

The Theseus Stack

Dialectic · *Noosphere* · *Theseus Codex*

A comprehensive architectural and algorithmic guide
to the firm's instrumentation for recorded reasoning

Scope. The three software surfaces of the Theseus monorepo: the desktop live-conversation analyzer (Dialectic), the Python reasoning and ingestion engine (Noosphere), and the Next.js web control plane and public publishing surface (Theseus Codex). Includes elevator pitches, full architecture, dataflow, algorithms, data models, code organization, and operational paths.

Reading order: §1 elevator pitches and the unified picture; §2–4 deep dive per component; §5 cross-cutting integration; §6 the methodological standard.

Contents

1	Orientation: what Theseus is, and why these three pieces	2
1.1	The firm’s premise in one sentence	2
1.2	The three surfaces	2
1.3	One picture of the whole stack	3
2	Dialectic: the live conversation analyzer	3
2.1	What a session feels like	4
2.2	The live pipeline, end to end	4
2.2.1	Why this shape works at <2 s latency	5
2.3	The interlocutor (“SP09”)	6
2.3.1	The quality gate	6
2.3.2	Logging	7
2.4	Module map	7
2.5	The wire format Noosphere expects	9
3	Noosphere: the reasoning and ingestion engine	9
3.1	What Noosphere is, conceptually	9
3.2	The classical seven-box pipeline (still real)	9
3.3	Data model: what every object actually is	10
3.3.1	Enums (the system’s vocabulary)	10
3.3.2	Core objects	11
3.3.3	Adversarial and methodology	12
3.3.4	Currents, Forecasts, governance	12
3.4	The ingestion pipeline	13
3.4.1	Binary extractors	13
3.4.2	Text-level pipeline	13
3.4.3	The principle-shape contract	13
3.5	The six-layer coherence engine	14
3.5.1	Layer S_1 : Formal consistency (NLI)	14
3.5.2	Layer S_2 : Argumentation theory	14
3.5.3	Layer S_3 : Probabilistic coherence	14
3.5.4	Layer S_4 : Embedding geometry	15
3.5.5	Layer S_5 : Information-theoretic compression	15
3.5.6	Layer S_6 : Citation-enforced LLM judge	15
3.5.7	Aggregation	16
3.6	Ontology graph (noosphere/ontology.py)	16
3.6.1	Distillation: claims $\rightarrow$ principles	16
3.6.2	Conviction	17

3.7	Inference: how a question is answered	17
3.8	Temporal tracking	17
3.9	Methodology profiles	18
3.10	Adversarial / red team — the “Dialectical Crucible”	18
3.11	Currents: the live event → public opinion loop	19
3.12	Forecasts: probability, calibration, paper and live bets	20
3.12.1	Generating a forecast	20
3.12.2	Resolution and calibration	20
3.13	Storage	21
3.14	Codex bridge	21
3.15	The CLI surface	21
3.16	Key design decisions, restated	21
4	Theseus Codex: the web control plane and public face	22
4.1	Two surfaces in one app	22
4.2	Stack and packaging	23
4.3	Authentication	23
4.3.1	Session flow	23
4.3.2	API keys (how Dialectic logs in)	23
4.4	The schema	24
4.4.1	Multi-tenant root	24
4.4.2	Auth and founders	24
4.4.3	Uploads (raw materials)	24
4.4.4	Conclusions and contradictions	24
4.4.5	Methodology system	25
4.4.6	Principles	25
4.4.7	Publication and reader response	25
4.4.8	Currents and forecasts	26
4.4.9	Source credibility and citation chain	26
4.4.10	Belief revision and drift	26
4.4.11	Subscribers, observability, social	26
4.5	The upload pipeline	26
4.5.1	Three execution paths, one route	27
4.6	The Currents proxy	27
4.7	Founder workspace pages (selected)	28
4.8	Public publication pipeline	29
4.9	The Electron desktop wrapper	29
4.10	Notable engineering choices	30
5	How the three pieces fit together	31

5.1	The journey of one upload	31
5.2	Shared abstractions across components	32
5.3	Operational summary	33
6	The methodological standard the software embodies	33
6.1	Five working criteria (from THE_META_METHOD.md)	33
6.2	Six standards for a recorded conclusion	34
6.3	Where the firm refuses to act	34
6.4	Closing observation	34

1 Orientation: what Theseus is, and why these three pieces

1.1 The firm's premise in one sentence

Theseus is a research and investment firm whose product is *recorded, inspectable reasoning*. The premise is methodological: a conclusion is only as valuable as the reasoning trail that produced it, and reasoning only compounds across time if a later reviewer can read the source material, the method, the objection, and the conditions of revision. The three software components in the monorepo are instruments for keeping that record honest at three different moments: **while** a discussion is happening, **when** written and recorded material is being processed, and **where** the firm's conclusions are reviewed, calibrated, and published.

1.2 The three surfaces

Dialectic — the live capture instrument

A PyQt6 desktop app. You launch it before a working session, log in to the Codex, and talk. It listens to the microphone with Silero voice-activity detection, transcribes utterances with a small faster-whisper model, and decomposes the running transcript into atomic claims. Pairwise NLI (a DeBERTa-v3 cross-encoder) scans every new claim against a rolling pool of recent claims and flags contradictions. The dashboard shows three panes — the running transcript, a force-directed claim graph, and a TENSIONS panel listing red-flagged pairs — and an opt-in *interlocutor* mode can interject in third person (“Theseus notes that the previous claim conflicts with...”) under a strict quality gate. Output is a JSON-Lines file (one finalized claim per line, with timestamp, speaker, text, MiniLM embedding, contradiction pair ids, topic cluster), and an optional reflection bundle. These files are the native input format for Noosphere.

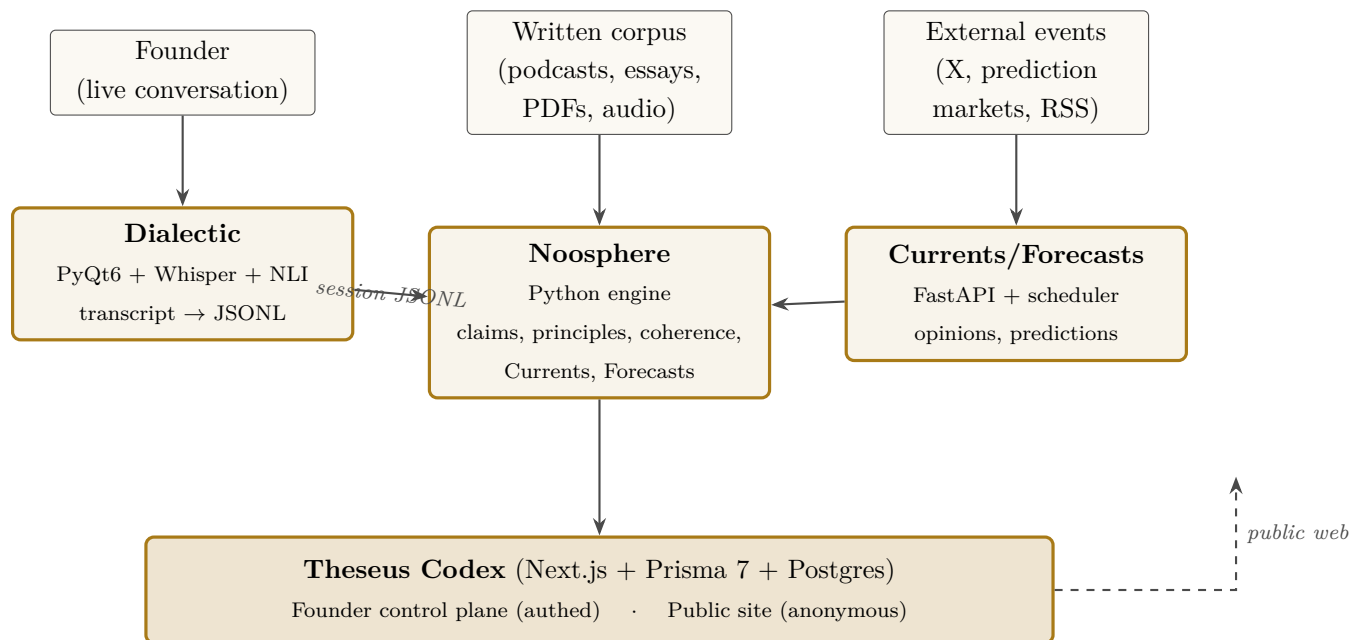
Noosphere — the engine

A large Python package. It ingests podcast transcripts, written essays, PDFs, audio, X posts, and prediction-market data; it extracts atomic claims, distills them into principles, and builds a knowledge graph in which nodes are principles and edges carry typed relations (SUPPORTS, CONTRADICTS, REFINES, INSTANTIATES, ...). It runs a six-layer coherence engine over those propositions (formal NLI, argumentation theory, probabilistic consistency, embedding-geometry sparsity, information-theoretic compression, and a citation-enforced LLM judge), generates adversarial objections against new conclusions, profiles each upload's reasoning *method* (not just its conclusions), tracks how principles drift across time, and runs two production pipelines: *Currents* (live X events → relevance-gated firm opinions → public articles) and *Forecasts* (Polymarket / Kalshi markets → firm probability predictions → Brier-scored calibration). It exposes a Typer CLI and writes its results back into the Codex database.

Theseus Codex — the control plane and public face

A Next.js 16 / React 19 / Prisma 7 web application against Supabase Postgres, deployed on Vercel; also packaged as an Electron desktop wrapper. Two surfaces in one app. The *public site* (theseuscodex.com) shows the firm's reviewed conclusions, published articles, Currents opinions, public forecasts, methodology pages, a calibration manifest, a critique submission form, and an Atom/RSS feed. The *founder control plane* ((`authed`)/ route group, gated by middleware) is roughly seventy pages covering uploads, transcript exploration, conclusion browsing, contradiction triage, knowledge graph and embedding-explorer views, founder-side Currents and Forecasts, the publication-review queue, principles triage, methodology review, API key minting (for Dialectic), and operations dashboards. The Codex is the durable read model for everything Noosphere produces and the surface where founders authorize live actions (publishing, live betting).

1.3 One picture of the whole stack



The flow you should read in this diagram: humans and the world produce raw material at the top; the three runtimes process it; and Postgres (read by the Codex) is the persistent shared substrate. *Dialectic is the front-most capture surface; Noosphere is the algorithmic spine; Codex is the durable view and publication face.*

2 Dialectic: the live conversation analyzer

2.1 What a session feels like

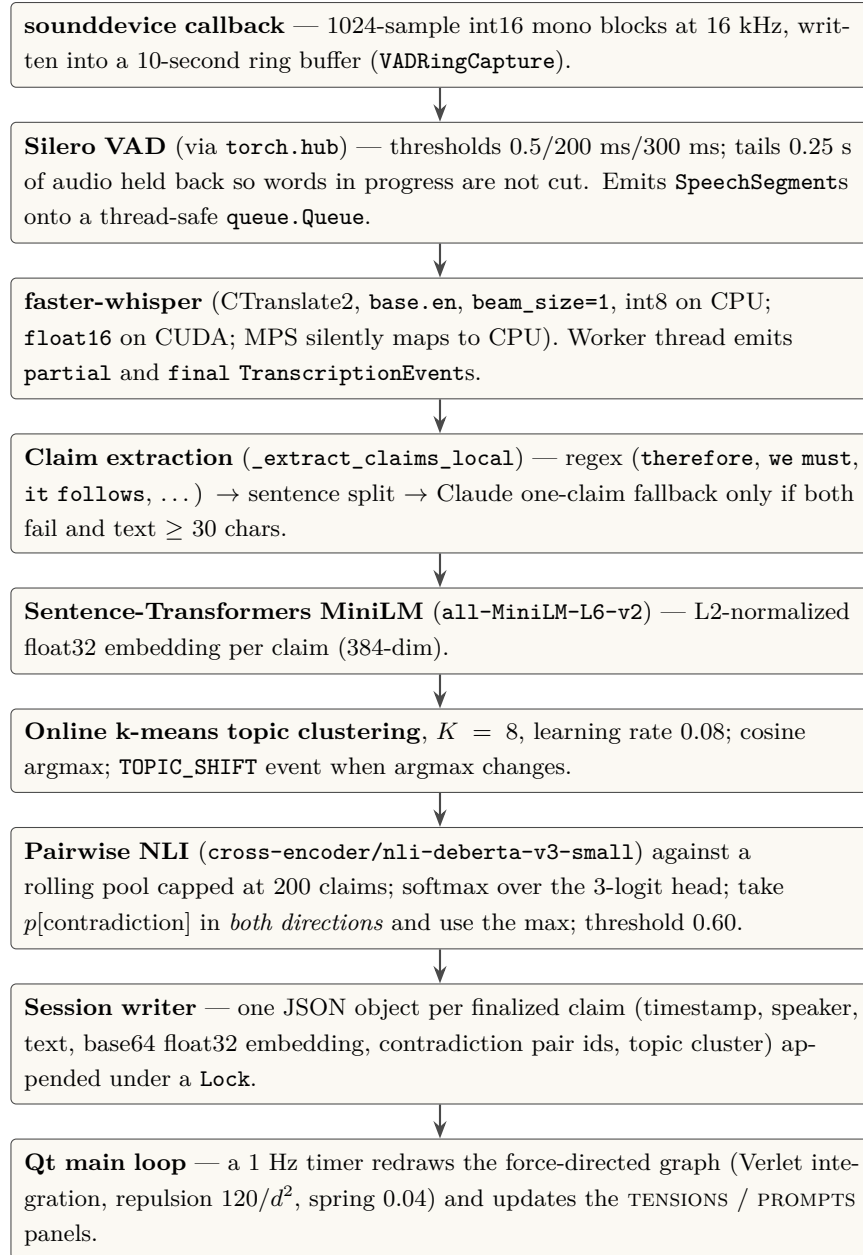
Open Dialectic. A login dialog appears — authentication against the Codex is mandatory, by design, because every analytical artifact will eventually be tied to a founder identity. You click `BEGIN SESSION` on the main window. The dashboard shows four panels:

- **Transcript pane**, streaming partial then finalized hypotheses from Whisper.
- **Claim graph**, a force-directed layout where each finalized claim is an amber dot labeled with a truncated quote, and contradictions are red edges.
- **tensions panel**, red-bordered cards: “claim A vs claim B, severity 0.74”.
- **prompts panel**, where the consent-gated Theseus interlocutor surfaces follow-up questions in third person.

When you click `STOP SESSION`, the app writes a `.wav` of the raw audio (kept local as provenance), a `.jsonl` of the analyzed claims, an `_interventions.jsonl` of every interlocutor candidate (whether emitted or dropped), and a single `_reflection.json` bundle. If the Codex credentials are configured, the JSONL and reflection bundle are POSTed to `/api/upload` after the session ends; audio is never auto-uploaded (Vercel’s 4.4 MB body cap, and a deliberate provenance posture).

2.2 The live pipeline, end to end

The full chain is constructed inside `ThreePaneDialecticWindow._start_session` in `dialectic/dashboard.py`. Each stage runs on its own thread or task; the UI thread does almost nothing except draw.



2.2.1 Why this shape works at <2 s latency

The README claims sub-2-second speech-to-claim latency. The shape that makes that possible is not GPU acceleration — it is staging. Each pipeline stage costs at most a few hundred milliseconds and runs in its own worker, with cross-thread plumbing via `queue.Queue` (PortAudio → transcriber), `asyncio.Queue` (transcriber → analyzer), and `asyncio.Queue` (analyzer → dashboard). The UI runs at 1 Hz; nothing heavy ever touches it.

An honest disclosure

The README mentions “MPS/GPU device selection”. In the code, `faster_whisper_device` explicitly maps `mps`→`cpu` because CTranslate2 has no MPS backend; only Silero VAD (a small torch model loaded via `torch.hub`) actually benefits from MPS. Whisper, MiniLM, and the cross-encoder all run on CPU. The “*int8* CTranslate2 on Apple Silicon CPU” is genuinely fast — but it is CPU, not GPU. Knowing this matters when you size hardware.

2.3 The interlocutor (“SP09”)

The interlocutor is a small piece of code (`dialectic/interlocutor.py`, ~450 lines, no Qt or model dependencies) that decides when Theseus should speak. Its design is conservative: by default it does nothing. The four modes are:

- SILENT (default for non-three-pane window) — analyzer fires but nothing is emitted.
- PASSIVE (default for three-pane) — overlay-only, no TTS.
- CONVERSATIONAL — overlay plus TTS after a 1.5-second pause.
- TUTOR — higher rate, smaller budget, requires explicit acknowledgment.

There are three intervention kinds: CONTRADICTION (fed by the analyzer’s NLI), OPEN_QUESTION (fed by an open conversational “loop” that has been hanging for ≥ 40 s), and PREDICTION_RESOLUTION (regex-fed when someone says “I predict / I bet / I forecast / I expect...”). Each candidate carries exactly four overlay lines and a TTS text capped at ~200 chars (so it can be spoken in ≤ 12 s).

2.3.1 The quality gate

Every candidate passes through a gate before it can be emitted:

- CONTRADICTION: allowed if confidence ≥ 0.92 , or in PASSIVE mode at ≥ 0.85 . Hedging filter: if both statements have a hedge density > 0.35 (count of “might / perhaps / could / possibly”), the candidate is dropped as `hedging_pseudo_contradiction`.
- OPEN_QUESTION: allowed in any non-silent mode (rationale `open_loop_heuristic`, confidence 0.55).
- PREDICTION_RESOLUTION: always allowed.

A per-session **budget** caps total time spent intervening: 420 s for CONVERSATIONAL, 180 s for TUTOR; each emission is billed at 12 s as a conservative upper bound. A separate **cooldown** of 45 s prevents bursts. Voice-channel kill phrases (“Theseus, stand down”, “Theseus, quiet”) flip a session flag mid-conversation.

2.3.2 Logging

Every candidate — emitted or dropped — is appended to `<session_id>_interventions.jsonl` as an `InterventionRecord` with its quality-gate verdict. `save_reflection_bundle` writes `<session_id>_reflection.jsonl` with the current mode, opt-in state, stand-down flag, and the full intervention history. This is the artifact a founder reviews post-session in the Codex.

2.4 Module map

All paths under `dialectic/dialectic/`:

Module	Purpose
<code>__main__.py</code>	Argparse (-model, -device, -legacy); calls <code>run_dashboard</code> .
<code>config.py</code>	Pure dataclasses: <code>AudioConfig</code> , <code>TranscriptionConfig</code> , <code>BatchTranscriptionConfig</code> , <code>AnalysisConfig</code> , <code>InterlocutorConfig</code> , <code>AutoTrimConfig</code> , <code>AutoTitleConfig</code> , <code>AutoUploadConfig</code> . Every threshold lives here.
<code>audio.py</code>	<code>SpeechSegment</code> , <code>SileroVADSegmenter</code> , <code>energy fallback</code> , <code>VADRingCapture</code> , <code>legacy AudioEngine</code> , offline WAV-replay path.
<code>transcriber.py</code>	<code>WhisperTranscriber</code> (legacy) and <code>SegmentQueueTranscriber</code> (modern, queue-driven). Device/compute-type mapping.
<code>analyzer.py</code>	<code>LiveAnalyzer</code> (legacy, DBSCAN topics, last-8 NLI window) and <code>DialecticSessionAnalyzer</code> (modern, online k-means, 200-claim NLI pool).
<code>interlocutor.py</code>	Mode + intervention machinery; quality gate; consent and budget; JSONL logging; reflection bundle.
<code>tts_sidecar.py</code>	<code>pyttsx3</code> with macOS <code>say -v Zarvox</code> fallback; <code>DIALECTIC_TTS</code> env gate.
<code>dashboard.py</code>	Two windows: legacy <code>DialecticDashboard</code> and modern <code>ThreePaneDialecticWindow</code> . The latter is the default.
<code>session_writer.py</code>	The JSONL wire format: <code>FinalizedClaimLine.to_jsonl</code> , base64 float32 helpers, thread-safe writer.
<code>cloud_uploader.py</code>	Async multipart POST of JSONL + reflection to <code><base_url>/api/upload</code> . Audio not auto-uploaded.
<code>credentials.py</code> , <code>login_dialog.py</code>	Codex auth; stored credentials in <code>~/Library/Application Support/Dialectic/credentials.json</code> at 0600.
<code>recording_modal.py</code> , <code>recording_pipeline.py</code>	The <i>batch</i> post-recording flow (trim → transcribe at <code>medium.en</code> → auto-title via Haiku → Supabase signed-URL upload). Distinct from the live flow.
<code>batch_transcriber.py</code>	Separate model cache for batch transcription, forced CPU.

2.5 The wire format Noosphere expects

```
{
  "timestamp": "2026-04-14T19:42:11.301845+00:00",
  "speaker": "Speaker 1",
  "text": "We should treat model drift as a first-class risk.",
  "embedding": "<base64 of float32 little-endian, 384-dim, L2-normalized>",
  "contradictions": ["uuid-a:uuid-b"],
  "topic_cluster_id": "topic_3"
}
```

The pair id is `f"{min(a,b)}:{max(a,b)}"` so the same pair has a stable identifier independent of which utterance came first. Noosphere's consumer is `noosphere.ingest_artifacts.ingest_dialectic_session_`. A round-trip test in `tests/test_dialectic_ingest_roundtrip.py` formally pins this contract.

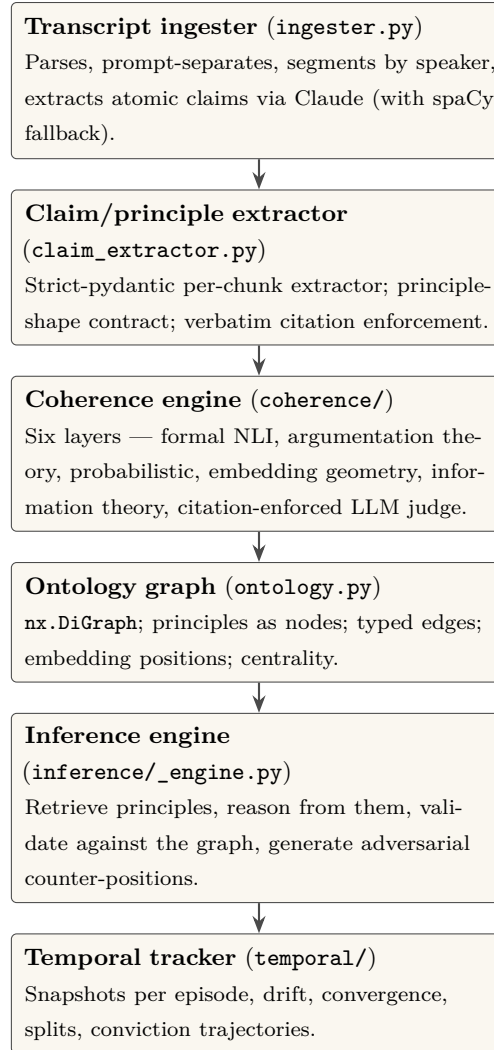
3 Noosphere: the reasoning and ingestion engine

3.1 What Noosphere is, conceptually

If Dialectic is a microphone and a vigilance system, Noosphere is the workshop where everything the firm reads, hears, and decides is broken apart, evaluated, related, and stored. Internally it is a Python package laid out as a strict dependency hierarchy. The CLI is the leaf; below it sit the subsystems (Currents, Forecasts, Literature, Benchmarks, Docgen); below those the *inquiry* layer (coherence, evaluation, peer review, red team, mitigations) and the *temporal* layer; below those the *methods* registry; below that the *io* layer (storage, codex bridge, ingestion); and at the leaf is *core* (models, the store, the ledger, the orchestrator, observability). The contract is enforced in CI by `lint-imports`.

3.2 The classical seven-box pipeline (still real)

The README's original diagram is still the right way to learn the spine:



But the code is broader than the diagram. The real `NoosphereOrchestrator.ingest_episode` in `orchestrator.py` routes claims into *two* stores: methodological claims feed the ontology graph; substantive claims feed a separate Conclusions Registry whose later outcomes *calibrate* the methods that produced them. This is the closed loop the firm cares about.

3.3 Data model: what every object actually is

The single fattest file in the codebase is `noosphere/models.py` (~2900 lines). It is a long list of Pydantic v2 `BaseModel` (transient) and `SQLModel table=True` (persisted) classes. The most load-bearing ones are described below.

3.3.1 Enums (the system's vocabulary)

- **RelationType**: SUPPORTS, CONTRADICTS, REFINES, INSTANTIATES, EXTENDS, ANALOGIZES, PRESUPPOSES, QUALIFIES.

- **ConvictionLevel:** AXIOM > STRONG > MODERATE > EXPLORATORY > CONTESTED.
- **Freshness:** FRESH, AGING, STALE, RETIRED.
- **CoherenceVerdict:** COHERE, CONTRADICT, UNRESOLVED.
- **PrincipleKind:** RULE, CRITERION, MECHANISM, HEURISTIC, DEFINITION, FORMULA, ALGORITHM (the seven shapes a principle-shaped Conclusion may take).
- **ClaimType:** FACTUAL, METHODOLOGICAL, NORMATIVE, PREDICTIVE, DEFINITIONAL, INTERPRETIVE.
- **ClaimOrigin:** FOUNDER, EXTERNAL, LITERATURE, ADVERSARIAL, VOICE, SYSTEM.
- **AdversarialChallengeStatus:** PENDING, EVALUATED, ADDRESSED, FATAL, SURVIVED, FALLEN.
- **CurrentEventStatus:** OBSERVED, ENRICHED, OPINED, ABSTAINED, REVOKED.
- **OpinionStance:** AGREES, DISAGREES, COMPLICATES, ABSTAINED.

3.3.2 Core objects

Claim The atomic unit. One sentence, one speaker. Fields include `text`, `speaker`, `episode_id`, `disciplines`, `embedding` (SBERT), `confidence`, `timestamp_seconds`, `source_id`, `source_span_*`, `voice_id`, `founder_id`, `author_key`, `effective_at`, `freshness`, `claim_type`, `claim_type_verified` (a separate NLI re-check), `claim_origin`, `evidence_pointers`, `confidence_hedges`.

Principle A distilled, durable belief. Carries `text`, `description`, `disciplines`, `conviction`, `conviction_score` (in $[0, 1]$), `embedding`, `supporting_claims`, `first_appeared`, `last_reinforced`, `mention_count`, and the six per-layer coherence scores plus a composite. Reappearances *update* a principle (never duplicate).

Conclusion A substantive, principle-shaped reasoned output. Carries `reasoning`, `rationale`, `kind` (FOUNDER/FIRM/METHOD/SYNTHESIS/ARTICLE), `confidence_tier` (OPEN/FOUNDER/FIRM/HIGH/MODERATE), `principles_used`, `evidence_chain_claim_ids`, `dissent_claim_ids`, plus the principle-shape contract: `principle_kind`, `domain_of_applicability` (≤ 300 chars), `quantifiable_proxies` (≤ 5), `decision_examples` (≤ 3), `source_span` (verbatim substring of the source chunk).

Relationship A directed graph edge. `source_id` $\rightarrow$ `target_id`, `relation`, `strength`, `evidence`, `detected_by` (extraction/coherence/geometric/manual).

SixLayerScore One per coherence evaluation: s_1 consistency, s_2 argumentation, s_3 probabilistic, s_4 geometric, s_5 compression, s_6 LLM-judge, each in $[0, 1]$.

ContradictionFinding A minimal pair: `id_a`, `id_b`, `severity`.

CoherenceReport Corpus-level summary with `composite_score`, per-layer means, contradictions, weakest links.

3.3.3 Adversarial and methodology

AdversarialChallenge Persisted objection against a Conclusion. Carries `cluster_fingerprint` (sha256 of principle + sorted evidence ids), `tradition` (the intellectual school the objection invokes), `primary_attack_vector`, `objection_text`, `atomic_claim_ids`, `prior_engagement`, `status`, the full six-layer score against the conclusion, and a possible `judge_overturned_contradict` flag.

MethodologyProfile A row describing *how* a source reasons, not what it concludes. Carries `pattern_type`, `title`, `summary`, JSON columns for `reasoning_moves`, `transfer_targets`, `assumptions`, `failure_modes`, `evidence_anchors` (sentence indices + verbatim quotes), `confidence`, `dedupe_key`.

3.3.4 Currents, Forecasts, governance

CurrentEvent Observed external post. Carries `source`, `external_id`, `text`, `captured_at`, `embedding`, `metrics` (XSignificanceMetrics: a weighted log-sum of impressions / retweets / likes / replies / quotes-and-bookmarks at weights 0.4/0.3/0.2/0.05/0.1), `status`.

EventOpinion The firm's source-grounded stance on an event. `stance`, `confidence`, `headline` (≤ 140 chars), `body_markdown`, `uncertainty_notes`, `abstention_reason`.

ForecastPrediction The firm's probability assignment to a Polymarket/Kalshi market. Carries `probability_yes`, `confidence_low`, `confidence_high`, `status` (PUBLISHED or one of several ABSTAINED_*), and a `live_authorized_at/by` that gates the paper-to-live crossing.

ForecastResolution Outcome + `brier_score` + `log_loss` + `calibration_bucket`.

Method, MethodInvocation, LedgerEntry, CascadeNode, CascadeEdge The Round-3 governance substrate: a versioned method registry, an Ed25519-signable hash-chained audit ledger, and an explicit provenance DAG where every edge is labeled with the method invocation that produced it.

DecayPolicy, RevalidationResult The freshness/decay engine. Policies are typed (FIXED_INTERVAL, EVIDENCE_CHANGED, METHOD_VERSION_BUMP, EMBEDDING_DRIFT, OUTCOME_OBSERVED, CALIBRATION_REGRESSION, ANY, ALL).

QuantitativeFormalisation The newest layer (prompt 63, 2026-05-15) requiring every *approved* principle to declare a `null_hypothesis`, ≥ 1 metric, and ≥ 1 test before publication.

3.4 The ingestion pipeline

3.4.1 Binary extractors

Audio inputs go through `AudioExtractor`, which calls `faster-whisper` locally (default `small.en`) and falls through to OpenAI Whisper if local fails or if `NOOSPHERE_FORCE_OPENAI_WHISPER=1`. PDFs go through `PdfExtractor`: `pypdf` extracts text page-by-page with `[page N]` markers; `digital_confidence` measures the fraction of pages with ≥ 80 chars of recovered text. If that fraction is below 25% and `NOOSPHERE_ENABLE_OCR=1`, the file goes to `ocrmypdf` (which requires tesseract + ghostscript). Otherwise it fails fast with a message telling the operator how to enable OCR.

3.4.2 Text-level pipeline

`TranscriptIngester.ingest` is the orchestrator. Roughly:

1. Read the file.
2. Run the `PromptSeparator` from the mitigations layer to strip interview prompts, debate setups, and quoted external positions *before* segmentation. The point is to prevent prompt material in transcripts from being treated as the founder’s own assertion.
3. Detect the format (timestamped, labeled `SPEAKER:`, or unstructured) and dispatch to the right parser.
4. Run `ClaimExtractor.extract` in batches of 7 segments. The primary path calls Claude with a strict JSON schema; the fallback (no API key) uses spaCy `en_core_web_sm` sentence splitting and keeps only sentences matching assertoric verb patterns (`is/are/was/were, can/could/will/would, ...`). `_is_valid_claim` actively rejects non-propositional patterns (“I think...”, “let’s...”, um, bare wh-questions).
5. For each LLM-flagged claim where `is_author_assertion=false`, set `claim_origin=external`. Downstream coherence, voice, and Codex sync filter those out.
6. Enrich each claim: apply ingestion flags, normalize the text (zero-width strip, whitespace collapse), embed with `SentenceTransformer` (`all-mpnet-base-v2` by default), classify into `Disciplines` via keyword match plus optional embedding prototype refinement.

3.4.3 The principle-shape contract

There is a second extractor at the chunk level: `PrincipleExtractor`. It enforces a hard contract on every principle-shaped row it emits:

- Text must not start with “I/we/my/our” (downgrades to the sentinel `NO_PRINCIPLE_EXTRACTABLE`).
- Text must be ≥ 12 characters.

- `principle_kind` must be one of the seven enum values.
- `source_span` must be a *verbatim* substring of the source chunk; citation drift drops the row.
- `domain_of_applicability` is truncated to 300 chars.
- `quantifiable_proxies` capped at 5; `decision_examples` at 3.

This contract is the spine of the post-pivot methodology: “every principle has a domain and a falsifiable test, or it does not get to be a principle.”

3.5 The six-layer coherence engine

This is the part most worth understanding deeply, because it is where the firm’s epistemic position is operationalized. There are two parallel implementations:

- **Corpus-level** (`coherence/engine.py::CoherenceEngine`) — runs all six layers over a whole set of propositions and emits a `CoherenceReport`.
- **Pair-level** (modules `nli.py`, `argumentation.py`, `probabilistic.py`, `geometry.py`, `information.py`, `judge.py`, combined by `aggregator.py`) — runs the six layers on one *pair* and emits a single `CoherenceVerdict`.

3.5.1 Layer S_1 : Formal consistency (NLI)

The corpus score is a weighted contradiction sum:

$$\text{WCS}(\Gamma) = \frac{\sum_{i < j} w_{ij} P(\text{CONTRADICT} | p_i, p_j)}{\sum_{i < j} w_{ij}}, \quad w_{ij} = \frac{1}{2}(\text{conv}_i + \text{conv}_j), \quad S_1 = 1 - \text{WCS}.$$

The NLI backend tries local DeBERTa-NLI first (`cross-encoder/nli-deberta-v3-base`) and falls back to Claude via a JSON-schema prompt that returns entailment / neutral / contradiction probabilities. The pair-level verdict is `CONTRADICT` when the contradiction probability is ≥ 0.55 and dominates entailment.

3.5.2 Layer S_2 : Argumentation theory

Builds a Dung-style directed attack graph in which A attacks B iff $P(\text{CONTRADICT} | A, B) > 0.5$, then computes the *grounded extension* of that graph by iteratively marking nodes with no incoming attack as accepted and removing them until fixpoint. $S_2 = |\text{accepted}| / |\text{nodes}|$. The pair-level verdict is `COHERE` iff both are jointly acceptable; `CONTRADICT` iff they mutually attack.

3.5.3 Layer S_3 : Probabilistic coherence

Two flavors. Corpus level: Roche’s measure $C_R(\Gamma) = \frac{1}{|\Gamma|^2} \sum_{i \neq j} (P(p_i | p_j) - P(p_i | \neg p_j))$, with the conditionals estimated from NLI; $S_3 = \sigma(C_R)$. Pair level: an LLM extracts each claim’s probabilistic

commitments (numbers, modal ranges like “likely $\rightarrow$ (0.55, 0.85)”) and `check_kolmogorov_for_pair` flags out-of-range probabilities and finite-additivity violations.

3.5.4 Layer S_4 : Embedding geometry

This is the firm’s distinctive empirical bet, validated in `ideologicalOntology/Contradiction_Geometry/`. The intuition: in standard sentence embeddings, “the sky is blue” and “the sky is not blue” have a cosine similarity of ~ 0.9 . Raw cosine therefore cannot distinguish disagreement from disagreement-about-the-same-topic. The fix is that *contradiction concentrates the difference vector $\mathbf{a} - \mathbf{b}$ into a small number of dimensions* — measured by Hoyer sparsity:

$$h(\mathbf{x}) = \frac{\sqrt{n} - \|\mathbf{x}\|_1 / \|\mathbf{x}\|_2}{\sqrt{n} - 1} \in [0, 1].$$

Higher h means the energy is concentrated in fewer dimensions. The pair-level rule (`coherence/geometry.py::score`) is:

$$\text{verdict} = \begin{cases} \text{CONTRADICT} & \text{if } h \geq \theta_h \text{ and } \frac{1}{2}(c_a + c_b) < \theta_c, \\ \text{COHERE} & \text{if } h < 0.5 \theta_h, \\ \text{UNRESOLVED} & \text{otherwise,} \end{cases}$$

with θ_h, θ_c loaded from the calibration JSON in `ideologicalOntology/Contradiction_Geometry/results/refin` if present (default (0.35, 0.35)).

The module `noosphere/geometry.py` goes further: a logistic regression on stacked known-contradictory and non-contradictory difference vectors produces a unit “contradiction direction” $\hat{c}$, and `project_onto_contradict` yields a continuous score. PCA over the same set of differences identifies a low-dimensional “contradiction subspace”. The `IdeologyReflector` applies the Householder reflection $\mathbf{v}' = \mathbf{v} - 2(\mathbf{v} \cdot \hat{a})\hat{a}$ across any chosen concept axis $\hat{a}$ — so reflecting a position across the class axis (computed as $\text{mean}(\text{positive terms}) - \text{mean}(\text{negative terms})$, normalized) produces its rhetorical dual. This is the technical underpinning of the firm’s “Reverse Marxism” research: the strongest case against X is the reflection of the strongest case for X .

3.5.5 Layer S_5 : Information-theoretic compression

The intuition: two propositions that share structure compress *better* together than separately. The pair score is

$$S_5 = 1 - \frac{|\text{zstd}(\text{text}_a || \text{text}_b)|}{|\text{zstd}(\text{text}_a)| + |\text{zstd}(\text{text}_b)|}.$$

The pair verdict thresholds on this ratio against an empirically calibrated $\theta = 0.92$.

3.5.6 Layer S_6 : Citation-enforced LLM judge

The judge is the most controversial layer because it is the place a generative model is asked for a verdict — exactly the place rationalization can leak in. The design counters this:

- The judge prompt receives the prior five layer scores by name (e.g. `s1_consistency: 0.4321, s4_geometric: 0.6712, ...`).
- The judge must produce a `LLMJudgeVerdictPacket` with `verdict`, `confidence`, `explanation`, `cited_prior_scores`.
- Post-processing (`explanation_cites_prior_layers`) verifies that each cited prior score appears in the explanation by name *and* by its numeric value to 1–3 decimals.
- The judge can override a CONTRADICT aggregate verdict to UNRESOLVED only if (a) the citation check passes, (b) the cited values match the prior scores, and (c) `judge_cited_scores_match_prior` from `mitigations/coherence_guard.py` confirms.

This is the firm’s anti-rationalization guard.

3.5.7 Aggregation

Corpus level uses a fixed weighted average: $s_1=0.25, s_2=0.10, s_3=0.15, s_4=0.15, s_5=0.10, s_6=0.25$.

Pair level uses majority of six — 4-or-more wins; otherwise UNRESOLVED. Platt scaling per layer

(`coherence/calibration.py`) keeps scores comparable. Per-domain recalibration (`coherence/recalibration.py`)

fires when enough new outcomes have landed.

3.6 Ontology graph (`noosphere/ontology.py`)

A `networkx.DiGraph` whose nodes are principles (and optionally claims, as leaves) and whose edges are typed `Relationships`. `OntologyGraph` stores three parallel dicts and exposes both queries (`get_related`, `get_supporting_claims`, `get_contradictions`, `find_nearest_principles`, `get_axioms`, `get_principle_neighborhood`) and analytics (density, conviction distribution, betweenness centrality).

3.6.1 Distillation: claims → principles

`PrincipleDistiller` is the cluster-and-distill step:

1. Embed every claim missing an embedding.
2. `AgglomerativeClustering(distance_threshold=0.3, linkage="average")` on cosine distance.
3. For each cluster, send to Claude (`claude-sonnet-4-20250514`) with a JSON schema demanding `{principle_text, description, conviction_level, disciplines}`.
4. Deduplicate by cosine similarity in the existing graph (threshold 0.85); on hit, update `mention_count`, `last_reinforced`, merge `supporting_claims`.
5. Add a `Relationship(SUPPORTS, strength=0.95)` edge from every supporting claim.

6. Detect inter-principle relations: $\text{cosine} > 0.8$ between two new principles yields a **REFINES** edge with `detected_by="geometric"`.

3.6.2 Conviction

The `ConvictionEstimator` (in `temporal/__init__.py`) computes

$$\text{conv} = 0.3\sigma(\text{mention freq}) + 0.2 e^{-\Delta_{\text{days}}/30} + 0.2\sigma(\text{support/mention}) + 0.3\sigma(\text{support}/10),$$

clamped to $[0, 1]$. The 30-day half-life is deliberate: a principle that hasn't been reinforced in a month is decaying.

3.7 Inference: how a question is answered

`InferenceEngine.ask(question, ...)` (`inference/_engine.py`):

1. Embed the question.
2. `PrincipleRetriever.retrieve` returns the top- k principles (default 10) by cosine, filtered by `conviction_score` ≥ 0.3 , then graph-traverses to depth 1 to pick up supporters/refiners.
3. For each retrieved principle, pull its supporting claims.
4. Send to Claude with principles as axioms and claims as evidence; demand a structured **ReasoningChain** (list of **ReasoningSteps**).
5. Validate the answer's coherence against the retrieved principles.
6. `AdversarialGenerator.generate_counter_position`: pick a concept axis (class by default), Householder-reflect the leaned-on principle's embedding across that axis, find the nearest principles to the reflected point, and have Claude synthesize the strongest counter-position from those neighbors. This is the systematized version of "what would a smart Marxist say about this?".
7. Run a final **ConsistencyChecker** guard.
8. Return an **InferenceResult** with caveats, principles used, and corpus-coherence number.

There is also an *inverse* inference engine (`inference/inverse.py`): given that event E happened, what does it imply about, or refute in, our corpus? It returns supporting and refuted implications plus a **BlindspotReport** listing missing entities, missing mechanisms, and adjacent empty topics — a structured "things the firm should have said but didn't".

3.8 Temporal tracking

`TemporalTracker` writes one **TemporalSnapshot** per principle per episode, capturing the conviction score, mention count, embedding, and `drift_from_origin = cosine_distance(current,`

first). `EvolutionAnalyzer` surfaces:

- `detect_emergence(episode)` — principles first seen here.
- `detect_abandonment(lookback, threshold)` — principles with no recent reinforcement.
- `detect_convergence(pid_a, pid_b)` — pairs whose similarity is trending up.
- `detect_splits(pid)` — sub-principles whose meaning has diverged.
- `ideological_velocity(window=5)` — mean recent drift across active principles.

A stance-drift series (`build_stance_embedding_series`) computes per-author/topic rolling-window centroids, takes velocities and accelerations, and a z-score test (`z_threshold=2.5`) emits `DriftEvents` with an LLM-generated natural-language summary and a pointer to the earliest inconsistent claim.

3.9 Methodology profiles

`noosphere/methodology.py` answers “how did this person reason?” rather than “what did they conclude?”. A `MethodPattern` is the code-side template for one reasoning pattern: `first_principles_decomposition`, `adversarial_revision`, `analogical_transfer`, `dialogic_unfolding`, and others. Each carries a one-sentence rule, a transfer condition, an assumption it presumes, and a failure mode.

`derive_methodology_profiles(text, source_title)` is the deterministic detector. It splits the text into sentences (30–600 chars), scores every sentence against every pattern’s `evidence_terms`, ranks patterns by aggregate hits, and for the top six emits a `MethodologyProfileDraft` with up to four anchor sentences as evidence, a confidence in [0.42, 0.92], transfer targets inferred by intersecting content words with domain term-sets, and the verbatim quotes pinned to sentence indices. These rows persist in the Codex’s `MethodologyProfile` table.

3.10 Adversarial / red team — the “Dialectical Crucible”

`noosphere/adversarial.py` implements the firm’s systematic objection cycle. For a Conclusion:

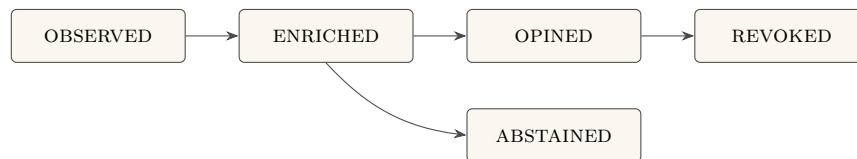
1. Compute `cluster_fingerprint = sha256(principle_id + sorted(evidence_ids))`. This is the stable handle for “this argument by this evidence”, before any particular wording.
2. `retrieve_prior_engagement` scans the last 400 claims for lexical overlap so the generator knows what the firm has already said.
3. Build the generator prompt: “ideologically neutral epistemic adversary; produce exactly three objections from three distinct intellectual traditions; each objection must include atomic_claims; flag citation_style as cited/synthesized/mixed; flag is_novel_vs_archive=false if the firm has likely already engaged.”
4. Call Claude with the schema (retry up to 3 times on parse failure).

5. Turn each objection into `Claim` rows with `claim_origin=adversarial` and a synthetic critic speaker.
6. Run the six-layer coherence aggregator on each objection *against* the conclusion's anchor claim and supporting evidence.
7. **finalize_survival_status**: if any objection lands a `CONTRADICT` verdict with confidence ≥ 0.65 , the conclusion *falls* (status `FALLEN`, tier ladder demoted: `FIRM` $\rightarrow$ `MODERATE` $\rightarrow$ `LOW` $\rightarrow$ `SPECULATIVE`).
8. Founders can apply a `HumanAdversarialOverride.kind` of `addressed` or `fatal`.

The economic point is that the firm tier-demotes conclusions automatically when a sufficiently strong objection lands — the adversarial system is enforcement, not just commentary. There is also a `redteam.py` that catalogs synthetic attacks against the pipeline itself (prompt injection in transcripts, polluted PDF text layers, citation fabrication, judge rationalization), each paired with a mitigation in `noosphere/mitigations/`.

3.11 Currents: the live event $\rightarrow$ public opinion loop

The Currents scheduler (`currents/scheduler.py`) runs every 300 seconds. The lifecycle is:



1. Ingest X posts; compute `XSignificanceMetrics.significance_score` (weighted log-sum); dedupe by cosine similarity.
2. Enrich (embed; mark near-duplicates).
3. Significance gate: below the threshold $\rightarrow$ `ABSTAIN_BELOW_SIGNIFICANCE_FLOOR`.
4. Relevance gate: retrieve against Conclusions and Claims; require ≥ 2 qualifying hits with score ≥ 0.55 ; otherwise `ABSTAIN_INSUFFICIENT_SOURCES` / `ABSTAIN_OFF_DOMAIN`.
5. Charge an hourly token budget; call Haiku with a prompt wrapped in `PROMPT_SEPARATOR` markers; parse JSON; **validate citations**: every quoted span must appear verbatim in a retrieved source. If not, `ABSTAINED_CITATION_FABRICATION`.
6. Write the `EventOpinion` and one `OpinionCitation` per supporting span.
7. Run a strawman-detection guard.
8. Every hour, `articles.triggers.dispatch_triggered_articles` drafts public articles from the most significant opinions, capped at the weekly article cap.

9. Embed-backfill runs daily; outbound social posts are held until a founder explicitly approves them.

3.12 Forecasts: probability, calibration, paper and live bets

The forecasts subsystem mirrors Polymarket and Kalshi markets, generates firm probability predictions, scores them after resolution, and runs paper or live bets.

3.12.1 Generating a forecast

`forecast_generator.generate_forecast:`

1. Retrieve ≥ 3 distinct sources for the market.
2. Refuse if it's within 1 hour of close, or if a near-duplicate forecast was published in the last 24 hours at cosine ≥ 0.92 .
3. Wrap the source bundle in prompt-separator markers.
4. Call the LLM with a strict JSON schema (`probability_yes`, `confidence_low/high`, `headline`, `reasoning_markdown`, `uncertainty_notes`, `citations`); up to 3 JSON-parse retries, 2 schema-validate retries, 2 citation-validate retries.
5. Run `validate_citations`; on fabrication, `ABSTAINED_CITATION_FABRICATION`.
6. Build a `MarketDecisionTrace` via `decision_metrics.build_decision_trace`. This is the *investable artifact*: a typed, fully deterministic record mapping inputs to an action (`HOLD`/`WATCH`/`PAPER`/`LIVE`) and a stake recommendation. No randomness. No further LLM calls. Versioned as `decision_metrics@v1`. Thresholds: edge ≥ 0.05 for paper, ≥ 0.08 for live; confidence ≥ 0.55 , locality ≥ 0.6 , liquidity ≥ 0.5 for live; contradiction veto at 0.5, decay veto at 0.9.

3.12.2 Resolution and calibration

On resolution: Brier score $(p - y)^2$, log-loss $-[y \ln p + (1 - y) \ln(1 - p)]$, and a calibration bucket (rounded probability) are computed and persisted. A founder can issue a `ResolutionOverride` if the venue is wrong; when the venue subsequently disagrees, a `ResolutionMismatch` is logged and the resolution is rolled forward with a `ResolutionRevision` that preserves prior settlement for audit.

A singleton `ForecastPortfolioState` per organization holds the paper and live balances, daily loss, and a kill switch that is consulted on every staking decision. The DB has a `CHECK` constraint that `ForecastBet.mode='LIVE'` requires `liveAuthorizedAt` IS NOT NULL — live betting is not a code check but a schema invariant.

3.13 Storage

`noosphere/store.py` (3683 lines) is the authoritative persistence layer over SQLite (dev) or Postgres (Supabase or Neon in production). It declares ~60 `SQLModel` tables and exposes a typed CRUD facade. Embedding vectors are stored as packed float32 in `LargeBinary` columns. The pool sizing is environment-aware: Supabase’s transaction pooler gets `poolclass=NullPool` (the pooler does the pooling); direct Postgres gets a sized pool; SQLite gets defaults with `check_same_thread=False`. SQLite-only ALTER TABLE migrations run on startup; Postgres uses Alembic.

A separate, older `conclusions.py::ConclusionsRegistry` persists substantive conclusions to `data_dir/conclusions_registry.json`. New code lands in `StoredConclusion`.

The `ledger/` subpackage adds a hash-chained, Ed25519-signable audit ledger with JSONL export and chain verification; the `cascade/` subpackage owns the provenance DAG.

3.14 Codex bridge

`noosphere/codex_bridge.py` is how the Vercel-hosted Codex talks to a Python runtime. Vercel cannot run Python, so uploads land as `Upload.status='queued_offline'` and a developer-machine cron processes them. `ingest_from_codex(upload_id)` connects to the Codex’s Postgres, reads the `Upload`, chunks the text, extracts claims (naive rule-based or LLM-based), detects contradictions, writes back `Conclusion` rows in the Codex schema, and stamps `Upload.status='ingested'` plus an `AuditEvent`.

3.15 The CLI surface

Two CLIs co-exist. The legacy Click CLI (`noosphere/cli.py`) exposes commands like `ingest`, `ask`, `graph`, `coherence`, `evolution`, `stats`, `search`, `contradictions`, `principles`, `calibration`, `conclusions`, `classify`, `ingest-written`, `founders`, `synthesis`, `research`, `forecasts`. The modern Typer CLI (`noosphere/typer_cli.py`, accessible as `python -m noosphere`) adds `synthesize`, `login`, `ingest-from-codex`, `codex-queued`, `codex-methodology-reanalyze`, `codex-transcript-enrich`, `codex-reanalyze`, `as-of` (time-travel), `diff`, `counterfactual`, `adversarial`, `ingest-voice`, `scoreboard`, `backup`, `restore`, and `evaluate`.

3.16 Key design decisions, restated

1. **Atomic propositions, not summaries.** The unit of analysis is the claim, not the paragraph. Multi-sentence agglomerations are actively rejected.
2. **Embeddings + difference vectors.** Raw cosine doesn’t distinguish “the sky is blue” from “the sky is not blue”. Difference-vector Hoyer sparsity does.
3. **Graph, not vector store.** Typed edges encode what coherence and inference actually need.

4. **Conviction weighting.** A contested principle attacked by an axiom carries different weight than the reverse.
5. **Temporal indexing.** Reappearance updates a principle; meaning drift is tracked even when the label is unchanged.
6. **Methodology / substance split.** Methodological claims feed the brain; substantive conclusions feed the Conclusions Registry, whose later outcomes calibrate the methods that produced them.
7. **Citation-enforced LLM judge.** The LLM is allowed to be a judge only when it can cite the five mechanical layers by name and number.
8. **Prompt separator + ingestion guard everywhere.** Every prompt sent to the model is wrapped in unambiguous markers; transcripts have prompt material stripped before segmentation.
9. **Verbatim citation contract.** Quoted spans must appear verbatim in the retrieved source. The failure mode is abstention, not retry-and-explain.
10. **Reproducible decision traces.** Forecasts publish a typed deterministic trace; the LLM’s prose explains, never overrides.
11. **Versioned method registry + cascade DAG.** When a method version bumps, everything it produced is automatically eligible for revalidation.
12. **Kill switch + paper-first.** Live betting requires explicit per-prediction authorization, enforced at the schema level.

4 Theseus Codex: the web control plane and public face

4.1 Two surfaces in one app

The Codex is a Next.js 16 (App Router) / React 19 / TypeScript application backed by Prisma 7 against Supabase Postgres in production, with an Electron desktop wrapper for offline use. It is both:

- the firm’s **public site** at `theseuscodex.com` — reviewed conclusions, published articles, public Currents opinions, public forecasts, methodology pages, a calibration manifest, a critique submission form, RSS/Atom feeds, and an “ask the firm” Q&A;
- the founders’ **private control plane** at `(authed)/` — about seventy pages covering everything from uploads to operations dashboards, gated by middleware.

4.2 Stack and packaging

From `package.json`: `next 16`, `react 19`, `prisma 7`, `@prisma/adapter-pg 7.7`, `pg 8.20`, `bcryptjs 2.4`, `tailwindcss 4`, `ioredis 5.6` (cloud job queue), `mammoth` (DOCX), `pdf-parse` (PDF), `openai 4` (Whisper for audio), `three 0.184` + `@react-three/fiber` + `drei` (embeddings explorer canvas), `react-markdown` + `remark-gfm` + `rehype-sanitize`, `electron 35` + `electron-builder 26`.

A README disagreement worth knowing

The README still says SQLite; the actual `prisma/schema.prisma` declares `provider = "postgresql"` and the codebase explicitly comments that SQLite was abandoned (Prisma 7 bakes the provider into generated migrations). Production is Supabase Postgres via the driver-adapter pattern. Local offline development can run a Docker Postgres or, for the Electron app, a per-user file-mode SQLite-compatible DB.

4.3 Authentication

Founder accounts are home-rolled (no Supabase Auth, no NextAuth). Supabase is only Postgres plus blob Storage for podcast audio.

4.3.1 Session flow

POST `/api/auth/login` takes `{email, password, organizationSlug, next}`. Identity match is permissive (lowercase email or username, scoped to the org). Password verification is `bcrypt.compare`; failed attempts go through `checkLoginRateLimit`. On success the server creates a `Session` row (30-day expiry, hex token) and sets cookie `theseus_session = b64url(JSON{token, exp}) . b64url(HMAC-SHA256(payload, SESSION_SECRET))`, `HttpOnly`, `SameSite=Lax`, `Secure` in production. The middleware (`src/middleware.ts`) does an Edge-safe presence check on every protected prefix; full HMAC + DB validation happens in server components.

Protected prefixes: `/dashboard`, `/codex-ask`, `/upload`, `/founders`, `/conclusions`, `/contradictions`, `/research`, `/open-questions`, `/publication`, `/library`, `/account`, `/forecasts/operator`, `/q/`.

Middleware also mints a `theseus_csrf` cookie (Web Crypto HMAC) on every authenticated request lacking one; mutating handlers verify it via `src/lib/csrf.ts`.

Password policy is enforced in `isStrongPassword`: ≥ 12 chars, mixed case + digit + symbol, not in a small common-passwords dictionary.

4.3.2 API keys (how Dialectic logs in)

Wire format `Authorization: Bearer tcx_<prefix>_<secret>`, where `<prefix>` is 12 base36 chars (plaintext, $O(1)$ lookup) and `<secret>` is 48 base36 chars hashed with `bcrypt` and stored in `ApiKey.keyHash`. Plaintext is shown to the founder *once*. `getFounderFromAuth(req)` tries API key first, then cookie; this is what `/api/upload` and `/api/auth/whoami` use.

There is intentionally *no signup endpoint*. Founders are seeded by `prisma/seed.ts` and added by `scripts/add-founder.ts`.

4.4 The schema

`prisma/schema.prisma` is the single most important file in the Codex — it is the durable read model for everything Noosphere produces. About 75 models grouped roughly as follows.

4.4.1 Multi-tenant root

Organization is the root; **slug** is the URL-routing key and the login scope. Every collection cascades from here.

4.4.2 Auth and founders

Founder (**role**: `admin/founder/viewer`, **noosphereId** bridges to Python-side **FounderProfile**); **Session** (server-side); **ApiKey** (machine-to-machine); **AuditEvent** (append-only ledger of every founder action); **DeletionRequest**, **ConclusionDeletionRequest** (peer-to-peer “please delete this” workflows); **DashboardDismissal** (per-founder hide preferences).

4.4.3 Uploads (raw materials)

Upload carries the title, description, source type (`written / annotation / external / transcript`), original filename, MIME, file path (real or `inline:<uuid>` on serverless), the extracted **textContent** (what Noosphere actually reads), extraction method, status (`pending / extracting / awaiting_ingest / processing / ingested / failed / queued_offline`), a **processLog** capped at 200 KB (live Python stdout streams into this), counts of resulting claims and methods and principles, and *public blog fields* (**publishedAt**, **slug**, **blogExcerpt**, **authorBio**, **audioUrl**, **audioDurationSec**). Visibility is a ladder (`org / semi-private / private`); soft-delete is **deletedAt**. **UploadChunk** stores chunked text with optional **startMs/endMs/speakerLabel/headingHint** for transcripts.

4.4.4 Conclusions and contradictions

Conclusion mirrors `noosphere.models.Conclusion`: **text**, normalized text (lowercased — the dedup key), **confidenceTier** (`firm / founder / open`), **rationale**, JSON arrays for supporting principles / evidence / dissent, **confidence**, **topic hint**, **embedding JSON**, and the principle-shape contract fields (**principleKind**, **domainOfApplicability**, **quantifiableProxies**, **decisionExamples**, **sourceSpan**).

ConclusionSource is the many-to-many bridge to uploads, with custom delete semantics: when an upload is soft-deleted, its bridge rows are hard-deleted; any orphan conclusion is hard-deleted too. This is what prevents a duplicate-text conclusion from disappearing when only one of its sources is gone.

Contradiction (severity, six-layer JSON, narrative, status, resolution), **OpenQuestion** (unresolved claim-vs-claim disagreements), **ReviewItem** (older coherence-engine review queue), **ResearchSuggestion** (“read X next”).

4.4.5 Methodology system

MethodologyProfile (the Noosphere methodology profile, persisted); **MethodologyQualityScore** (one-to-one with conclusion; operationalizes the five working criteria from `THE_META_METHOD.md`: progressivity, severity, aim-method-fit, compressibility, domain-sensitivity, plus a non-mean composite); **ConclusionMethod** (which registered methods produced this conclusion at what weight); **MethodVersion** (content-addressed snapshots of method source + rationale + failures yaml + domain bound); **MethodTrackRecord** (materialized per-method per-domain Brier + calibration slope + severity pass rate); **MethodRetirement** (`active` / `under_review` / `deprecated` / `retired` state machine); **AnchorRevision** (versioned k-medoid centroids defining a method’s domain in embedding space); **DomainBoundVerdict** (per-(conclusion, method) verdict: `in_bounds` / `edge_case` / `out_of_bounds`); **CalibrationModel** (domain-scoped isotonic regression, refit weekly).

4.4.6 Principles

Principle (corpus-level distillation output: status `draft` / `accepted` / `rejected` / `merged` / `needs_rereview`, conviction, domain breadth, cluster centroid similarity, public visibility); **QuantitativeFormalisation** (bridges a principle to a falsifiable test — `nullHypothesis`, metrics, tests, data sources, decision thresholds).

4.4.7 Publication and reader response

PublicationReview (the founder-side gate: `queued` / `in_review` / `needs_revision` / `declined` / `published`, checklist JSON for meta-analysis OK, adversarial-engaged OK, clarity OK, no leakage OK); **PublishedConclusion** (the public snapshot; immutable per (`slug`, `version`) — republishing bumps version; kind is `CONCLUSION` or `ARTICLE`; `discountedConfidence` for calibration display; full payload JSON); **PublicationSignature** (Ed25519 signature over canonical hash; *private keys never live in the web app* — signing happens in Noosphere); **Addendum** (dated, never edits the original body); **PublicResponse** (structured reader response with `publishConsent` for symmetric double opt-in); **ResponseTriage** (Python-classifier sidecar: `SUBSTANTIVE_OBJECTION` / `CLARIFICATION_REQUEST` / `GENERAL_ENGAGEMENT` / `SPAM_NOISE`); **PublicReply** (founder reply with snapshot of consent); **CritiqueSubmission** (invited expert critique with severity rubric); **CritiqueBountyPayout** (default \$500, requires founder confirmation — Codex never sends money, only records intent).

4.4.8 Currents and forecasts

The Codex side of Currents and Forecasts mirrors the Noosphere SQLModel tables: `CurrentEvent`, `EventOpinion`, `OpinionCitation`, `FollowUpSession`, `FollowUpMessage`; `ForecastMarket`, `WatchedMarket`, `ForecastPrediction`, `ForecastTrace`, `ForecastCitation`, `ForecastResolution`, `ResolutionOverride`, `ResolutionMismatch`, `ResolutionRevision`, `ForecastBet` (with the `CHECK` constraint enforcing live authorization), `ForecastPortfolioState` (singleton per org — bankroll, daily loss, kill switch, rolling 90-day Brier and log-loss).

4.4.9 Source credibility and citation chain

`SourceStanding` (append-only ledger: `ACTIVE` / `RETRACTED` / `CORRECTED` / `DISPUTED` / `EXPIRED`); `SourceTriageItem` (when a standing flips, one row per affected conclusion); `SourceCredibilityUpdate` (time series with running posterior alpha/beta); `CitationVerdict` (NLI judge’s verdict per (citation, computation): `relation` $\in$ `SUPPORTS/CONTRADICTS/QUALIFIES/MENTIONS`, `relationHolds` $\in$ `ENTAILS/CONTRADICTS/NEUTRAL/AMBIGUOUS`, verbatim excerpt and stated claim).

4.4.10 Belief revision and drift

`RevisionEvent` (every committed founder-confirmed revision; `planId` is a sha256 over inputs and thresholds; `preConfidenceSnapshot` is the before-state for time replay; `revertedAt` is non-null if undone); `DriftEvent` (two flavors: principle embedding drift and nightly per-method calibration drift, with severity `ok` / `warn` / `escalate` / `insufficient` and the sigma/p-value/slope); `AttentionAction` (snooze/dismiss/unsnooze in the unified attention queue).

4.4.11 Subscribers, observability, social

`Subscriber` (scope `firm` / `methodology` / `domain` / `conclusion`; cadence `weekly` / `immediate` / `monthly`; double opt-in); `DigestSend` with `ackTokenHash` (SHA-256 of a one-time token — no tracking pixels); `DigestAck` (“I read this” clicks); `SubscriberBounce`. `Span` (distributed trace), `MethodMetricRollup` (p50/p95/error rate), `AlertRule/AlertEvent`. `SocialPost` (*requires explicit founder approval before posting*). `MethodologyReviewWeek`, `MethodologyReviewDaySummary` (quarterly five-day review with Ed25519-signed day summaries).

4.5 The upload pipeline

The most consequential API route is `POST /api/upload`. Its job: accept a multipart upload, persist it, and *somehow* hand the work off to Noosphere.

1. **Auth.** `getFounderFromAuth` accepts cookie or `tcx_` API key. 401 if missing. 403 if `role=viewer`.
2. **Parse multipart.** `file`, `title`, `description`, `sourceType`, `publishAsPost`, `visibility`.

3. **Validate.** Allowlist extensions (`.md/.txt/.vtt/.jsonl/.json/.pdf/.docx/.mp3/.m4a/.wav/.webm/.ogg`)
file size ≤ 4.4 MB (Vercel Hobby body cap, enforced even on self-host for predictability).
4. **Extract text.** `extractText`: UTF-8 pass-through; `pdf-parse` for PDFs; `mammoth` for DOCX;
OpenAI Whisper for audio. NUL bytes are stripped (Postgres rejects them).
5. **Audio storage.** If audio, push to Supabase Storage at `<uuid>/<sanitized_filename>`; the
public URL becomes `Upload.audioUrl`. Files over 4.4 MB use the signed-URL flow instead.
6. **Persist raw bytes.** Locally to `./uploads` when not on Vercel; `inline:<uuid>` marker on
Vercel because the FS doesn't persist.
7. **Create the row.** Status `pending`, `processLog` seeded.
8. **Audit.** Write an `AuditEvent`.
9. **Async dispatch** (non-awaited): first fire a GitHub `repository_dispatch` (the `noosphere-process-uploads`
workflow), then `enqueueIngestJob(upload.id)` which either `lpushes` to Redis (if `REDIS_URL`
and `USE_JOB_QUEUE=1`) or calls `processUpload` in-process.
10. **Respond.** Immediately return the new row id, status, and metadata. The user gets the ack
while the background work continues.

`processUpload` itself: flip status to `processing`; re-attempt extraction if text is empty; detect whether Python is available; if not (Vercel), append a “handed off to GitHub Actions” or “run `process-codex-upload.sh` manually” note and set status `queued_offline`; if so, `runNoospherePython(["-m", "noosphere", "filePath", onChunk: appendLog])` so stdout streams live into `processLog`; then `noosphere synthesize`; on completion, set status `ingested`.

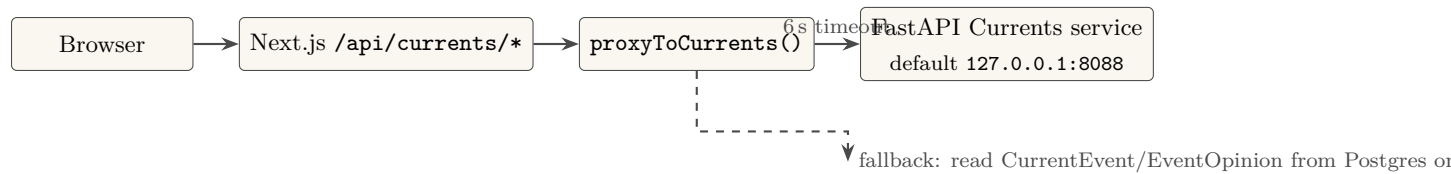
4.5.1 Three execution paths, one route

The route is environment-aware. The three paths are:

- **Pure local.** Spawn Python in-process. This is the README's path.
- **Self-hosted with worker.** Web app `lpushes` to Redis; `worker:ingest` pulls and runs `processUpload`.
- **Vercel (no Python).** Web app fires GitHub `repository_dispatch`. A workflow runs `noosphere ingest-from-codex` against the Supabase Postgres. A 10-minute cron sweep is the fallback.

4.6 The Currents proxy

`src/lib/currentsApi.ts` is the proxy core. The architecture is:



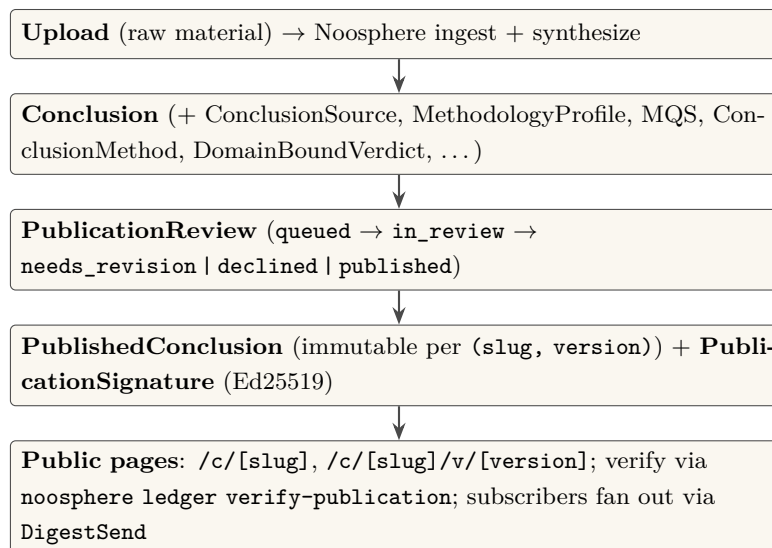
`proxyToCurrents` reuses the inbound `req.signal`, sets a 6-second upstream timeout, and pipes the body through (`duplex: "half"` for streaming). On 5xx, `currentsFallbackResponse` reads the Codex’s own Postgres tables directly — so the public Currents page degrades gracefully when the FastAPI service is down. The same pattern is replicated for forecasts (`forecastsApi.ts`).

4.7 Founder workspace pages (selected)

- `/dashboard` — operator home. Round-21 made principles the spine: “Active principles” is the top rail (accepted, conviction-ranked), then recent conclusions filtered by per-founder dismissals, recent uploads, drift events, reviewer-agreement telemetry. Banner area has an auto-processing status banner. Pulls Currents health, embedding health, forecast portfolio surface, decay records.
- `/library` — org-wide upload inventory. Owners delete their own; peers open a deletion request.
- `/upload`, `/upload/[id]` — form + per-upload detail with live `processLog` viewer.
- `/transcripts/[uploadId]` — chunked transcript with timestamps and speaker labels.
- `/conclusions`, `/conclusions/[id]` — sortable browser; detail shows attribution, tier, principle-shape fields, the MQS pill, source uploads, lineage links.
- `/contradictions`, `/open-questions` — severity-sorted active items.
- `/research`, `/reading-queue` — the research advisor’s surface.
- `/explorer` — the embeddings 3D explorer. `ExplorerCanvas` (R3F) renders points and edges via WebGL; saves views in URL-encoded state; supports hotkeys; shows empty-state diagnostics when the embedding index is unbuilt.
- `/knowledge` — graph view.
- `/founder-currents`, `/founder-currents/[id]` — founder-side feed with edit / revoke / edge badges.
- `/forecasts/setup`, `/forecasts/new`, `/forecasts/operator`, `/forecasts/portfolio` — operator surface includes the kill switch and live-bet authorization.

- `/publication` — review queue. Approve → mint `PublishedConclusion` + `PublicationSignature` → revalidate homepage tags.
- `/principles/queue`, `/principles/[id]/triage`, `/principles/[id]/quantitative` — accept / reject / edit distilled principles; attach quantitative formalizations.
- `/methods`, `/methods/[name]/[version]`, `/methods/candidates`, `/methods/graph` — method registry browser and dependency graph.
- `/methodology-review-week` — quarterly five-day review. Each day has a draft body and a founder-written body; the founder Ed25519-signs each day; once signed, the row is immutable until the signature is cleared.
- `/account/api-keys` — mint / revoke API keys. Plaintext shown once. This is where Dialectic and CLI keys come from.
- `/ops`, `/ops/ci`, `/ops/load`, `/ops/retention` — observability dashboards.

4.8 Public publication pipeline



For an article (longform essay), the same `PublishedConclusion` model is used with `kind="ARTICLE"`. For a blog post, the path is shorter: `Upload.publishedAt` + `slug` is the publish marker; `POST /api/publish` flips it. Visibility gates: `semi-private` and `private` are hard-rejected. Citations on public surfaces are gated to public sources only — `SourceStanding` cascades hide retracted citations, `CitationVerdict` blocks publication when the NLI judge finds entailment fails.

4.9 The Electron desktop wrapper

The repo also ships an Electron app called “Theseus Founder Portal” (`appId: com.theseus.founder-portal` in `electron-builder.yml`, a relic of the earlier “founder-portal” name). The flow:

1. Single-instance lock.
2. Resolve DATABASE_URL to `file:<app.userData>/data/founder-portal.db`.
3. Run `prisma migrate deploy` against that DB (idempotent on every launch).
4. fork the `.next/standalone/server.js` on a free local port.
5. Render in a `BrowserWindow` with `contextIsolation: true, nodeIntegration: false`, hidden inset title bar on macOS.
6. Auto-update via `electron-updater` against the GitHub releases.

The desktop app does *not* bundle Noosphere. It expects either a working Python install at `../noosphere/` (local dev) or that the user is connected to the shared Supabase Postgres so ingest runs centrally via GitHub Actions.

4.10 Notable engineering choices

1. **Postgres-only Prisma.** The README's SQLite story is fiction; production is Supabase Postgres via Prisma 7 driver adapters.
2. **Edge-safe middleware, Node-safe handlers.** The middleware does presence + format only (`crypto.subtle`); full HMAC + DB validation happens in server components.
3. **Three ingest paths in one route.** `/api/upload` doesn't know whether it's on Vercel, self-hosted, or local — it always (a) tries GitHub dispatch, (b) calls `enqueueIngestJob` which picks Redis vs in-process.
4. **Streamed Python stdout into the DB.** The `processLog` column is the simplest possible live-progress UI.
5. **Soft delete + cascade rules are app logic, not FK CASCADE.** `Upload.deletedAt` is soft-delete; `Conclusion` lifecycle is custom (orphans are hard-deleted).
6. **Founders are never hard-deleted** (Round-18 audit) — relations use Prisma's default `ON DELETE RESTRICT`.
7. **Symmetric double opt-in for public replies:** snapshot at confirm time so a later consent revoke cannot rewrite the audit record.
8. **Signing keys never live in the web app.** `PublicationSignature` rows are written by `noosphere ledger sign-publication`; the Codex only stores the artifact.
9. **No tracking pixels, hashed senders, voluntary acks.**
10. **Custom CSRF, sessions, rate limiter, URL aliasing.** All in `src/lib/`.

11. **MQS composite is intentionally not a mean.** `domainSensitivity` is a multiplicative gate; a strong domain track record cannot flatter a weak-domain confidence.

5 How the three pieces fit together

5.1 The journey of one upload

To make the integration concrete, follow a single 4-hour podcast transcript from microphone to public article.

1. **Capture.** The discussion is recorded inside Dialectic. Silero VAD segments audio; faster-whisper transcribes each segment; the analyzer extracts atomic claims, embeds them with MiniLM, NLI-flags contradictions, and writes `session_<ts>.jsonl` plus `_reflection.json`.
2. **Hand-off.** `cloud_uploader` POSTs the JSONL and reflection to `<codex>/api/upload` with the founder's API key. The Codex creates an `Upload` row, audit-logs it, and kicks off async dispatch.
3. **Processing.** On Vercel, GitHub Actions runs `noosphere ingest-from-codex <upload_id>`. On a developer machine, the Redis worker or the in-process job runs the same. Either way, Python opens the Codex Postgres, reads `textContent`, chunks it, extracts claims (with the prompt separator and the spaCy fallback if the LLM is unavailable), embeds them, and writes `Conclusion`, `ConclusionSource`, `Contradiction`, `OpenQuestion`, and `MethodologyProfile` rows back to the Codex schema.
4. **Coherence and adversarial.** The orchestrator runs the six-layer coherence engine, generating `SixLayerScores`; new conclusions enter the adversarial cycle which generates objections from three intellectual traditions, evaluates them against the conclusion, and demotes the conclusion's tier if any objection lands a `CONTRADICT` verdict at confidence ≥ 0.65 .
5. **Drift and lineage.** Temporal snapshots update; drift events surface if the embedding has moved; cascade edges record the method invocations that produced every node.
6. **Founder review.** The founder lands on `/dashboard`, sees the new conclusion, opens `/conclusions/[id]`, reviews the methodology profile, MQS, sources, contradictions, and lineage. If publishable, they push it to `/publication`.
7. **Publication.** On approve, the Codex mints a `PublishedConclusion` (with frozen `payloadJson`), and Noosphere's ledger signs it (Ed25519, kept out of the web app). Public pages `/c/[slug]` and `/c/[slug]/v/[version]` become reachable; subscribers are fanned out via `DigestSend`.
8. **Currents loop.** An X post about a related event lands in `CurrentEvent`. The Currents scheduler enriches it, relevance-gates against the firm's conclusions (this freshly published one

is now retrievable), generates an opinion with cited spans, validates citations verbatim, writes `EventOpinion` + `OpinionCitation` rows. The article-dispatch trigger eventually clusters several opinions into a public article.

9. **Forecasts loop.** The same conclusion may be retrieved when a relevant Polymarket market is queued; the forecast generator builds a `MarketDecisionTrace` and (if it crosses paper threshold) places a paper bet. On resolution, Brier and log-loss are computed and the calibration manifest updates.
10. **Reanalysis.** Some weeks later, extraction logic changes. `scripts/reanalyze-codex.sh` -`apply` idempotently re-runs the chunks / methodology / embeddings / principle-shape passes against existing uploads. Old conclusions are updated, not wiped.

5.2 Shared abstractions across components

- **Claim.** The same atomic-proposition shape is used in Dialectic’s JSONL, in Noosphere’s `Claim`, and (loosely) in the Codex’s `Conclusion` fields. The roundtrip test in `dialectic/tests/test_dialectic_ingest` pins the contract.
- **Embedding format.** 384-dim float32 (MiniLM) for live capture; 768-dim float32 (mpnet) for canonical Noosphere ingest. Both are stored as packed bytes in `LargeBinary` columns or as JSON arrays in the Codex.
- **Citation as verbatim substring.** Every layer that emits a citation (Noosphere principle extractor, Currents opinion generator, forecast generator, public article) enforces that the quoted span must literally appear in the source. Failure is abstention.
- **Prompt separator markers.** The mitigations layer wraps every LLM input with unambiguous `<<PROMPT_SEPARATOR_..._BEGIN>>` / `_END` markers.
- **Versioned methods.** `Method` + `MethodInvocation` + cascade graph + `MethodVersion` (Codex) + `DecayPolicy.METHOD_VERSION_BUMP` together implement the property that bumping a method version makes every artifact it produced eligible for revalidation without wiping production.
- **Ed25519 ledger.** The publication signature, the methodology review week day summary, and the Noosphere ledger all share the same Ed25519 signing path, owned by Noosphere and *never* in the web app.

5.3 Operational summary

Surface	Runtime / deployment
Dialectic	PyQt6 desktop app, distributed as <code>.dmg</code> (macOS) and <code>.exe</code> (Windows) installers, built fresh from main via the rolling-release workflow.
Noosphere CLI	<code>python -m noosphere</code> , distributed both as <code>.tar.gz</code> and <code>.exe</code> ; also runs in CI via the GitHub Actions <code>noosphere-process-uploads.yml</code> workflow.
Noosphere Currents/Forecasts schedulers	Long-lived Python processes (Dockerfiles: <code>Dockerfile.api</code> , <code>Dockerfile.scheduler</code>). The FastAPI service serves <code>/v1/currents/*</code> .
Theseus Codex (web)	Next.js 16 deployed on Vercel; Postgres on Supabase; Storage on Supabase. Auto-redeploy on every push to <code>main</code> .
Theseus Founder Portal (desktop)	Electron wrapper around the standalone Next.js build, per-user file DB, auto-updated.

6 The methodological standard the software embodies

The system is animated by a position taken explicitly in `METHODOLOGICAL_REORIENTATION.md` (May 2026): *Theseus's product is not its conclusions; it is a method for arriving at conclusions in a way that survives review.* Every architectural decision in the stack maps to this stance.

6.1 Five working criteria (from `THE_META_METHOD.md`)

1. **Progressivity** — a method should generate claims that could have been wrong in observable ways.
2. **Severity** — the test should plausibly have exposed the claim's weakness if the claim were false.
3. **Aim-method fit** — a method should fit the question being asked, not the question one wishes were being asked.
4. **Compressibility** — explanations that require many special exceptions transfer worse.
5. **Domain sensitivity** — no method is reliable everywhere; methods have failure modes and boundary conditions.

In the Codex these five criteria are not abstract: they are columns on `MethodologyQualityScore`, with a non-mean composite that uses `domainSensitivity` as a multiplicative gate.

6.2 Six standards for a recorded conclusion

1. Name the conclusion.
2. Name the method that produced it.
3. Preserve the source trail.
4. Preserve the strongest objection.
5. State what would revise the conclusion.
6. Revisit the record when later evidence or outcomes arrive.

These six map directly to objects in the stack: `Conclusion`, `ConclusionMethod` + `MethodVersion`, `ConclusionSource` + cascade DAG, `AdversarialChallenge` (with status `SURVIVED/FALLEN`), the `Conclusion.principle_kind` contract's `domainOfApplicability/quantifiableProxies` fields, and `ForecastResolution` + `RevisionEvent`. The system is designed so that a conclusion lacking any of these six is, structurally, not a publishable artifact.

6.3 Where the firm refuses to act

Several places in the code are *designed* to abstain rather than continue. They are worth noting because they are where the methodology bites:

- **Currents.** If retrieval finds fewer than 2 qualifying conclusions at score ≥ 0.55 for an event, the system emits an `EventOpinion` with stance `ABSTAINED` and reason `INSUFFICIENT_SOURCES` or `ABSTAIN_OFF_DOMAIN`. The firm publishes a clear “we cannot comment”, not a thin opinion.
- **Citation fabrication.** If any quoted span doesn't appear verbatim in a retrieved source, the result is `ABSTAINED_CITATION_FABRICATION`. No retry-and-explain.
- **Forecasts close-to-market.** Within 1 hour of close, the engine refuses to predict.
- **Live bets.** The DB CHECK constraint refuses any live bet without per-prediction authorization.
- **Public articles.** `PublicationReview` declines a draft when the checklist fails; `SourceStanding=RETRACTED` hides cited sources; `CitationVerdict=ENTAILS` is required for the relation to publish.
- **Adversarial fall.** A conclusion attacked at confidence ≥ 0.65 is tier-demoted automatically.

6.4 Closing observation

The firm's bet is that, at scale, the value of a research-and-investment shop comes from a verifiable answer to the question “how did you get there?” more than from the answer itself. Dialectic, Noosphere, and the Codex are the three places that question gets answered: Dialectic captures the

discussion so the record exists; Noosphere processes the record so it can be reasoned over; and the Codex exposes the record — to the founders for review, and to the world for inspection.

This guide reflects the state of the codebase at the time of writing. The README files in each component are point-in-time documents; the source of truth is the code. Where the README and the code disagreed, the code was taken as authoritative and the disagreement was noted.